

Recursive Chart Learning: Admitted Path Traces, Evaluation, and Update

Author: Artur Motruk

Affiliation: Independent researcher

ORCID: 0009-0003-6928-1701

Programme: Relation-First Organization Programme

Document role: Core foundation / learning, path-trace, and update layer

Public release status: Repository manuscript for public programme release

Foundation Statement

Recursive Chart Learning Theory is a companion layer to Charted Organization Theory. It defines learning as **observer-relative path tracing of admitted branches through a chart-visible path field**.

Learning is not identical to improvement. A learning path trace records landed, admitted chart transitions. The evaluated outcome of a trace-admitted transition may be improving, neutral, worsening, or incomparable. The distinction is between success as a path-step and success as improvement. Every trace-admitted step succeeds as a path-step: it lands in an admissible endpoint and passes the declared admission condition. It may fail as improvement.

A path has no dangling arrow. A transition attempt that does not land in another admissible chart may be an attempted transition, but it is not a trace-admitted learning step. A path trace contains only landed and admitted transitions. A proper path trace also carries a task-declared threshold policy rather than an ad hoc threshold tuple.

Evaluation is not oracular. A task-evaluator may evaluate only chart-visible available steps or finite branches, or data explicitly declared by the task. It may not compare a proposed step to a globally unavailable outcome as if that outcome were already visible in the observer-frame.

Calibration is not a separate kind of path. Calibration is stability of task-evaluation over a chart-visible branch field. Repeated measurements, alternate branches, and comparison trials are all handled as available branches or branch-pairs under a declared comparison rule. Calibration unavailable, calibration unstable, task inadmissibility, and primitive path absence are distinct failure modes.

A task is not replaced by a selector, evaluator, calibrator, or loss. Selector, evaluator, constraint, query, loss, calibration, and update are task-modes. None is a new primitive.

Learning is path-sensitive. Two processes with the same endpoint chart need not be the same learning process, because their path traces may differ. Branching and re-merging are allowed. The same endpoint can be reached by different branches with different evaluation histories.

Compression is task-relative. A path trace compression may be lossless for a declared task while losing distinctions irrelevant to that task. Absolute losslessness over all possible chart-visible tasks is forbidden for a proper sub-chart unless the sub-chart becomes equivalent to the parent presentation.

The title term **recursive** is used in a specific formal sense. A learning path trace may update the chart or the available step field from which later branches are drawn. Recursion is not merely the existence of a finite sequence; it is the dependence of later chart-availability on earlier admitted path traces.

This paper makes no claim about biological cognition, language, artificial intelligence, physics, spacetime, or consciousness. Those may be later realizations. Here learning is defined only as a formal structure over charted organization.

0. Scope and Relation to Prior Layers

0.1 Scope

This paper defines a formal learning layer over the following prior structures:

1. a primitive relation carrier R ;
2. admissible charts $\mathcal{C}$ internal to R ;
3. chart-relative proper organizations $\text{Org}_\theta^\circ(\mathcal{C})$;
4. observer atlases $\mathfrak{A}_R = (\mathbf{C}_R, \mathbf{T}_R)$;
5. relation-internal tasks;
6. admissible chart transitions.

No physical realization map is defined here.

No claim is made here about brains, language, LLMs, adaptive organisms, dynamics in physical time, probability, optimization, thermodynamics, quantum theory, spacetime, or physical law.

0.2 Layer Separation

The Theory of Organization defines organization from a relation carrier.

Charted Organization Theory defines organization as chart-relative and atlas-relative.

Recursive Chart Learning Theory defines learning as trace-admitted movement through the chart-visible branch field of an atlas under task-relative evaluation and admission. The formal term is **path trace**. The word **ledger** may be useful informally, but is not used as a primitive term here.

The base question is not

What is learning psychologically?

but

When is a locally available branch evaluated, admitted, and recorded as a learning path trace?

0.3 Retained Commitments

The following commitments are retained.

1. Relation is prior to independently given answer, question, and value faces.
2. Organization lives in the proper non-boundary layer.
3. Charts are relation-maps internal to R , not external coordinate objects.
4. Observers are admissible positioned charts, not external subjects.
5. Information is chart-relative delineation.
6. Tasks are relation-internal demands, not external goals imposed without preservation data.
7. Selectors are task-induced chart selections, not new primitives.
8. Objectivity, when defined, is atlas-relative invariance under declared chart transitions.

0.4 New Commitments

This layer introduces the following commitments.

1. Learning is trace-admitted movement through chart-visible branches.
2. Learning is not identical to improvement.
3. Worsening, neutrality, and incomparability are explicit possible outcomes.
4. Evaluation is a task-mode acting on available steps or branches.
5. A path trace step must land in another admissible chart.
6. A path trace records only admitted landed transitions.

7. Branching and re-merging are allowed.
8. Calibration is stability of evaluation over task-comparable branches.
9. Local failure does not imply global path nonexistence.
10. Compression is task-relative.
11. Absolute lossless compression by a proper sub-chart is forbidden under complete task separation.
12. Endpoint equality is not learning equality; path traces matter.
13. Recursion means path-trace-induced change in the later available step field, not merely a finite sequence of transitions.
14. Proper path traces use task-declared threshold policies, not freely chosen threshold tuples.

0.5 Adjacent Frameworks and Import Boundaries

This paper is adjacent to several established literatures, but it does not import their technical machinery as axioms.

1. In reinforcement learning and Markov decision processes, learning is often formalized through states, actions, policies, rewards, and trajectories. The present paper is broader and weaker: charts are not states, branches are not assumed Markovian, and evaluators are not assumed to be rewards. The shared structural point is trajectory-sensitive evaluation.
2. In Bayesian decision theory, an inference target, loss or utility, and update rule are declared together. The task-mode structure below is a non-probabilistic analogue of that discipline. No Bayesian probability calculus is assumed.
3. In proper-scoring-rule and forecast-calibration theory, calibration is tied to comparison of predicted and observed quantities. Here calibration is defined more abstractly as stability of evaluation across task-comparable branches.
4. In rate-distortion and information bottleneck frameworks, compression is assessed relative to a declared distortion or relevance variable. Here compression is assessed relative to task-visible delineations. No entropy or mutual information structure is assumed, although entropy-like quantities may instantiate a declared branch-complexity functional in particular applications.
5. In operational reconstructions, admissible operations and available comparisons constrain what can be asserted. The no-oracle condition below is a charted-organization version of that operational discipline: evaluation may use available branches or declared task data, not hidden outcomes.
6. In philosophy of learning and epistemology, learning is often connected to improvement, knowledge, justification, error, or inquiry. This paper isolates a narrower formal structure: admitted, evaluated, trace-admitted branch movement. Improvement is a possible outcome, not the definition.

0.6 Branch Discovery Scope

The document assumes an observer atlas and its available step fields. It does not explain how the atlas is generated, how genuinely new branches enter the atlas, or how a domain discovers options not already present in its declared charted structure. Those are branch-discovery questions. The present paper begins once a charted field of available branches has been declared.

1. Inherited Data and Conventions

1.1 Primitive Carrier

Let R be a primitive relation carrier.

1.2 Observer Atlas

Let

$$\mathfrak{A}_R = (\mathbf{C}_R, \mathbf{T}_R)$$

be an observer atlas, where $\mathbf{C}_R$ is a class of admissible charts internal to R , and $\mathbf{T}_R$ is a declared class of admissible chart transitions.

1.3 Charts

For each chart $\mathcal{C} \in \mathbf{C}_R$, write

$$\mathcal{C} = (C, A_{\mathcal{C}}, Q_{\mathcal{C}}, Z_{\mathcal{C}}, \alpha_{\mathcal{C}}, \chi_{\mathcal{C}}, \zeta_{\mathcal{C}}, \preceq_{\mathcal{C}})$$

where $C \subsetneq R$ is the chart carrier, the maps

$$\alpha_{\mathcal{C}} : C \rightarrow A_{\mathcal{C}}, \quad \chi_{\mathcal{C}} : C \rightarrow Q_{\mathcal{C}}, \quad \zeta_{\mathcal{C}} : C \rightarrow Z_{\mathcal{C}}$$

are face-readout maps, and

$$(Z_{\mathcal{C}}, \preceq_{\mathcal{C}}, z_{\mathcal{C}}^-, z_{\mathcal{C}}^+)$$

is a bounded ordered zero-boundary face.

When the chart is functional, the induced resolution map is

$$\rho_{\mathcal{C}} : A_{\mathcal{C}} \times Q_{\mathcal{C}} \rightarrow Z_{\mathcal{C}}.$$

1.4 Proper Organization Layer

For $\theta \in Z_{\mathcal{C}}^{\circ}$, write

$$\text{Org}_{\theta}^{\circ}(\mathcal{C})$$

for the proper non-boundary organization layer of $\mathcal{C}$ at threshold θ .

1.5 Admissible Transitions

A chart transition

$$\phi : \mathcal{C} \rightarrow \mathcal{D}$$

belongs to $\mathbf{T}_R$ only when it is one of the declared admissible transition types in the atlas. This paper assumes that each such transition has enough structure to specify its source and target and to compose when path trace composition is invoked.

1.6 Order Notation

A preorder is denoted $\preceq$. The strict part of a preorder is denoted $\prec$:

$$x \prec y \iff x \preceq y \text{ and not } y \preceq x.$$

For an outcome preorder $(W_{\tau}, \preceq_{\tau})$, the notation

$$w_1 \prec_{\tau} w_2$$

means that w_2 is strictly above w_1 according to the declared outcome ordering.

2. Tasks and Task-Modes

2.1 Relation-Internal Task

A relation-internal task τ is a declared relation-internal demand. At minimum, it determines a set of task-visible distinctions to be preserved or resolved.

Write

$$D_\tau \subseteq R \times R$$

for the distinction demand of τ . If $(r, s) \in D_\tau$, then r and s must remain delineated for the task.

2.2 Task-Mode

A task-mode is an operational role of a task.

This paper uses seven modes.

1. Query mode: what must be resolved.
2. Selector mode: which chart or chart family can resolve it.
3. Constraint mode: which organizations count as admissible for the task.
4. Evaluator mode: which steps or branches are admitted and how outcomes are classified.
5. Calibration mode: which task-comparable branches are stable within tolerance.
6. Loss mode: how failure of preservation or resolution is measured.
7. Update mode: how a chart is changed after evaluation.

These are not additional primitives. They are modes of τ .

2.3 Selector Mode

Let

$$\text{Res}(\tau) = \{\mathcal{C} \in \mathbf{C}_R : \mathcal{C} \text{ resolves } \tau\}$$

be the resolving chart class of τ .

The selector mode of τ is the task-induced selection

$$\tau \mapsto \text{Res}(\tau)$$

or, when a declared preorder $\preceq_{\text{ch}}$ on charts is present, the minimal resolving chart class

$$\text{MinRes}(\tau) = \min_{\preceq_{\text{ch}}} \text{Res}(\tau).$$

2.4 Task-Visible Step and Branch Modes

The evaluator mode of a task does not evaluate a globally pre-known answer. It evaluates chart-visible available steps or finite branches.

Thus the evaluator mode must declare a domain of available branches:

$$X_\tau \subseteq \text{Br}_{\mathfrak{A}}^{<\infty},$$

where $\text{Br}_{\mathfrak{A}}^{<\infty}$ is the class of finite branches in the atlas, defined in Section 3.

2.5 Evaluator Mode

An evaluator mode of τ is a map

$$E_\tau : X_\tau \rightarrow W_\tau$$

where $(W_\tau, \preceq_\tau)$ is a preorder.

No evaluator appears without a declared task τ .

2.6 Admissible Evaluation Region

The evaluator mode declares a subset

$$W_\tau^{\text{adm}} \subseteq W_\tau$$

called the admissible evaluation region.

A step or branch whose evaluation lies outside W_τ^{adm} may be an attempted branch, but it is not trace-admissible as a τ -learning branch.

2.7 Outcome Classes

The evaluator mode may also declare outcome regions

$$W_\tau^+, \quad W_\tau^0, \quad W_\tau^-, \quad W_\tau^?$$

inside W_τ^{adm} , interpreted respectively as improving, neutral, worsening, and incomparable outcome classes.

Unless an overlap convention is explicitly declared, these classes are taken to be pairwise disjoint. They need not be exhaustive unless explicitly declared. If they are not exhaustive, the outcome classification is partial.

2.8 Calibration Mode

A calibration mode of τ consists of:

1. a comparison domain

$$\text{CalDom}_\tau \subseteq X_\tau \times X_\tau;$$

2. a comparison loss or distance

$$\text{CalLoss}_\tau : \text{CalDom}_\tau \rightarrow L_\tau;$$

3. an admissible tolerance region

$$L_\tau^{\text{cal}} \subseteq L_\tau.$$

Calibration compares task-comparable branches. It is not a separate path type.

2.9 Loss Mode

A loss mode of τ is a map

$$\text{Loss}_\tau : X_\tau \rightarrow L_\tau$$

where $(L_\tau, \preceq_L)$ is an ordered loss space. Lower loss is interpreted as no worse only when this ordering is explicitly declared.

A loss mode may induce an evaluator mode by order reversal or by a declared admissibility threshold.

A loss mode is **faithful to the task-visible delineations** D_τ when zero loss implies preservation of every distinction required by D_τ . Formally, if $p \in X_\tau$ is a branch or compression candidate, then

$$\text{Loss}_\tau(p) = 0 \implies p \text{ preserves every declared distinction in } D_\tau.$$

The faithfulness condition is not automatic. It is the bridge between a numerical or ordered loss and the task-visible distinctions that the task is supposed to protect.

2.10 Task Is Not Identical to Any One Mode

A task is not identical to its selector mode, evaluator mode, calibration mode, constraint mode, query mode, update mode, or loss mode.

A task may have any subset of these modes declared.

A branch can be evaluated as learning only when an evaluator mode, or an equivalent loss/admission mode, is declared.

2.11 Loss-Induced Evaluator

If a loss mode $\text{Loss}_\tau : X_\tau \rightarrow L_\tau$ and an admissible loss region $L_\tau^{\text{adm}} \subseteq L_\tau$ are declared, then an evaluator mode may be induced by

$$E_\tau^{\text{Loss}}(p) = \text{Loss}_\tau(p)$$

with

$$W_\tau^{\text{adm}} = L_\tau^{\text{adm}}.$$

If L_τ is ordered so that lower loss is better, improvement is read by order reversal. This induced evaluator is a task-mode; it is not an additional primitive.

2.12 Calibration-Compatible Evaluator

An evaluator mode E_τ is compatible with a calibration mode when every comparable pair $(p, q) \in \text{CalDom}_\tau$ has evaluations that can be compared by the calibration loss. In symbols,

$$(p, q) \in \text{CalDom}_\tau \implies (E_\tau(p), E_\tau(q)) \text{ is in the domain of the declared calibration comparison.}$$

Calibration is therefore not an extra mechanism above evaluation. It is a declared comparison rule over evaluations of task-comparable branches.

2.13 Update-Admissible Mode

An update mode of τ may assign an updated chart, atlas, or step field after a path trace has been admitted. It is **admission-dependent** when it is defined only on admitted path traces. Thus an update mode may have the form

$$U_\tau : \text{Tr}_\tau^{\text{adm}} \rightarrow \text{UpdateData}.$$

This keeps update downstream of admission. A non-admitted branch may be an attempted update, but it does not generate a trace-admitted learning update.

Theorem 2.14 — Faithful Loss Supports the Compression Boundary

If Loss_τ is faithful to D_τ , then zero loss for τ implies preservation of all task-visible delineations demanded by τ .

Proof This is exactly the faithfulness condition in Definition 2.9.

3. Local Step Fields and Branches

3.1 Available Step Field

For a chart $\mathcal{C} \in \mathbf{C}_R$, the atlas-available one-step field is

$$\text{Step}_{\mathfrak{A}}(\mathcal{C}) = \{(\phi, \mathcal{D}) : \mathcal{D} \in \mathbf{C}_R, \phi : \mathcal{C} \rightarrow \mathcal{D} \in \mathbf{T}_R\}.$$

This is the local field of chart-visible next moves from $\mathcal{C}$.

3.2 Task-Visible Step Field

A task τ may restrict the available step field to a task-visible field

$$\text{Step}_\tau(\mathcal{C}) \subseteq \text{Step}_{\mathfrak{A}}(\mathcal{C}).$$

If no restriction is declared, the task-visible field is not assumed equal to the atlas-available field. It must be declared.

3.3 Landed Step

Every element

$$(\phi, \mathcal{D}) \in \text{Step}_{\mathfrak{A}}(\mathcal{C})$$

is a landed step from $\mathcal{C}$ to $\mathcal{D}$.

A dangling attempt

$$(\mathcal{C}, \psi, *)$$

is not an element of $\text{Step}_{\mathfrak{A}}(\mathcal{C})$.

3.4 Branch

A branch is a source-tagged finite composable sequence

$$p = (\mathcal{C}_0; \phi_0, \mathcal{C}_1; \dots; \phi_{n-1}, \mathcal{C}_n)$$

usually displayed as

$$\mathcal{C}_0 \xrightarrow{\phi_0} \mathcal{C}_1 \xrightarrow{\phi_1} \dots \xrightarrow{\phi_{n-1}} \mathcal{C}_n.$$

For every $i < n$,

$$(\phi_i, \mathcal{C}_{i+1}) \in \text{Step}_{\mathfrak{A}}(\mathcal{C}_i).$$

Thus each branch carries its source chart, target chart, and transition sequence as part of its data. Composability is not automatic for arbitrary formal sequences; it is guaranteed only when every step is present in the declared atlas transition class $\mathbf{T}_R$.

The source of p is $\text{src}(p) = \mathcal{C}_0$. The endpoint is $\text{tgt}(p) = \mathcal{C}_n$. The length is $|p| = n$.

3.5 Branch Field

The finite branch field from $\mathcal{C}$ is

$$\text{Br}_{\mathfrak{A}}^{<\infty}(\mathcal{C}) = \{p : p \text{ is a finite branch from } \mathcal{C}\}.$$

The depth-limited branch field is

$$\text{Br}_{\mathfrak{A}}^{\leq n}(\mathcal{C}) = \{p \in \text{Br}_{\mathfrak{A}}^{<\infty}(\mathcal{C}) : |p| \leq n\}.$$

The global finite branch class is

$$\text{Br}_{\mathfrak{A}}^{<\infty} = \bigcup_{\mathcal{C} \in \mathbf{C}_R} \text{Br}_{\mathfrak{A}}^{<\infty}(\mathcal{C}).$$

This union is a disjoint union at the level of branch data, because every branch is source-tagged by $\text{src}(p)$.

3.6 Task-Visible Branch Field

The task-visible branch field is

$$\text{Br}_{\tau}^{<\infty} = X_{\tau} \subseteq \text{Br}_{\mathfrak{A}}^{<\infty}.$$

A task-visible branch is a branch on which the task evaluator is defined.

3.7 Branching and Re-Merging

Two branches $p, q \in \text{Br}_{\mathfrak{A}}^{<\infty}(\mathcal{C})$ branch when they share a source chart and differ at some later chart or transition.

They re-merge when they have the same endpoint chart, or when their endpoint charts are declared organization-equivalent.

Branching and re-merging are permitted.

3.8 Branch Equivalence

A task τ may declare an equivalence relation

$$p \sim_\tau q$$

on a subset of X_τ . Such branches are task-comparable.

Task-comparable branches are the natural domain for calibration.

3.9 Branch Neighborhood

A task-visible branch neighborhood at $\mathcal{C}$ is a declared subset

$$\mathcal{N}_\tau(\mathcal{C}) \subseteq \text{Br}_\mathfrak{A}^{<\infty}(\mathcal{C}).$$

Evaluation, selection, calibration, and admission may be performed on such a neighborhood rather than on a single preselected branch. This is the local non-oracular comparison field available to the observer-chart for task τ .

4. Evaluation, Admission, and No-Oracle Constraint

4.1 Evaluated Branch

A branch $p \in X_\tau$ is evaluated by

$$E_\tau(p) \in W_\tau.$$

4.2 Admitted Branch

A branch $p \in X_\tau$ is admitted when

$$E_\tau(p) \in W_\tau^{\text{adm}}.$$

4.3 Admitted Step

A one-step branch

$$p = (\mathcal{C} \xrightarrow{\phi} \mathcal{D})$$

is an admitted step when it is admitted as a branch.

4.4 Learning Path Trace

A τ -learning path trace is an admitted branch selected from X_τ .

Equivalently, a learning path trace is a finite landed chart path whose branch evaluation lies in W_τ^{adm} .

When stepwise admission is declared, each one-step subbranch must also be admitted. Stepwise admission is an additional condition, not automatic.

If both whole-branch admission and stepwise admission are declared, then a branch is trace-admissible only when both conditions hold. Whole-branch admission evaluates the path as a composite. Stepwise admission evaluates each local step. Neither condition follows from the other without an explicit compositional assumption on the evaluator.

4.5 Outcome Value

For a τ -learning path trace p , write

$$w_\tau(p) = E_\tau(p).$$

4.6 Improving, Neutral, Worsening, and Incomparable Path Traces

A τ -learning path trace p is improving when

$$w_\tau(p) \in W_\tau^+.$$

It is neutral when

$$w_\tau(p) \in W_\tau^0.$$

It is worsening when

$$w_\tau(p) \in W_\tau^-.$$

It is incomparable when

$$w_\tau(p) \in W_\tau^?.$$

4.7 Successful Learning

Successful learning is improving admitted learning.

Successful learning is a subtype of learning, not the definition of learning.

4.8 Regressive Learning

Regressive learning is worsening admitted learning.

The framework does not remove or disclaim regressive learning. It is one of the possible evaluated outcomes. What makes the transition learning is that it is available, admitted, evaluated, and trace-admitted. What makes it regressive is the direction of its evaluator outcome. Directionality classifies learning; it does not decide whether the admitted path trace is learning at all.

4.9 Successful Branch Versus Successful Improvement

Every trace-admitted learning branch is successful as a path: it lands through admissible charts and passes the task-admission condition.

A trace-admitted learning branch may fail as an improvement: it may be worsening, neutral, or incomparable.

4.10 Chart-Internal Evaluator

An evaluator $E_\tau : X_\tau \rightarrow W_\tau$ is chart-internal to the atlas when both conditions hold:

1. its domain is chart-visible,

$$X_\tau \subseteq \text{Br}_{\mathfrak{A}}^{<\infty};$$

2. its computation depends only on branch data in X_τ and on explicitly declared task data.

Declared task data may include a comparison criterion, a target region, a loss rule, or a branch-neighborhood restriction. Undeclared external outcomes do not count as task data.

Theorem 4.11 — No-Oracle Evaluation

An evaluator is chart-internal if and only if it satisfies both chart-visible domain and declared-dependency conditions of Definition 4.10. In particular, an evaluator that classifies a branch by comparing it to an undeclared branch, outcome, or target outside the chart-visible branch field is not chart-internal.

Proof The first sentence is Definition 4.10. If an evaluator depends on an undeclared comparison target outside the chart-visible branch field, then either the target is outside the domain or the computation uses data not declared by the task. In either case one of the two conditions fails. Conversely, if both conditions hold, the evaluator uses only chart-visible branches and declared task data, which is precisely chart-internality.

Corollary 4.12 — Evaluation Requires Available Alternatives or Declared Criteria

An evaluator may compare branches only when the alternatives are available in the branch field or when the comparison criterion is explicitly declared as part of the task.

5. Calibration as Branch-Stability

5.1 Calibration Domain

For a task τ , the calibration domain is

$$\text{CalDom}_\tau \subseteq X_\tau \times X_\tau.$$

If $(p, q) \in \text{CalDom}_\tau$, then p and q are task-comparable for calibration.

5.2 Calibration Loss

A calibration loss is a map

$$\text{CalLoss}_\tau : \text{CalDom}_\tau \rightarrow L_\tau$$

with a declared calibration-admissible region

$$L_\tau^{\text{cal}} \subseteq L_\tau.$$

5.3 Calibration-Stable Pair

A comparable pair $(p, q) \in \text{CalDom}_\tau$ is calibration-stable when

$$\text{CalLoss}_\tau(p, q) \in L_\tau^{\text{cal}}.$$

5.4 Calibration-Unavailable Pair

A pair (p, q) is calibration-unavailable when

$$(p, q) \notin \text{CalDom}_\tau.$$

Calibration-unavailable does not mean calibration-failed.

5.5 Calibration-Failed Pair

A comparable pair $(p, q) \in \text{CalDom}_\tau$ fails calibration when

$$\text{CalLoss}_\tau(p, q) \notin L_\tau^{\text{cal}}.$$

Theorem 5.6 — Calibration Unavailability Is Not Calibration Failure

If

$$(p, q) \notin \text{CalDom}_\tau,$$

then no calibration-stability or calibration-failure statement is defined for (p, q) under τ .

Proof Both calibration stability and calibration failure require the value $\text{CalLoss}_\tau(p, q)$. This value is defined only on CalDom_τ . If $(p, q) \notin \text{CalDom}_\tau$, the comparison is unavailable, not failed.

5.7 Chart Calibration

A chart $\mathcal{C}$ is $(\tau, \text{CalLoss}_\tau)$ -calibrated over a branch class $Y \subseteq X_\tau \cap \text{Br}_\mathfrak{A}^{<\infty}(\mathcal{C})$ when every declared comparable pair in Y is calibration-stable.

5.8 Calibration Is Not a Separate Path Type

Calibration is a property of task-evaluation over comparable branches. It is not a separate kind of chart transition.

6. Observer-Relative Availability and Failure Modes

6.1 Primitive Path Availability

If a global path structure on R is declared, write

$$\text{Path}_R(x, y)$$

for the primitive paths from x to y .

This paper does not assume that such a global path structure is always declared.

6.2 Chart-Visible Path Availability

For a chart $\mathcal{C}$, write

$$\text{Path}_\mathcal{C}(x, y)$$

for the paths from x to y visible in $\mathcal{C}$, when such a path notion is declared.

When both notions exist, chart-visible paths are at most primitive paths exposed by the chart:

$$\text{Path}_\mathcal{C}(x, y) \subseteq \text{Path}_R(x, y).$$

6.3 Atlas-Visible Path Availability

A path from x to y is atlas-visible when it is visible in at least one chart of $\mathbf{C}_R$.

6.4 Task-Admitted Path Availability

A branch is task-admitted when it lies in X_τ and its evaluation lies in W_τ^{adm} .

6.5 Path-Complete Chart

A chart $\mathcal{C}$ is path-complete for a task τ when every τ -relevant primitive path is chart-visible in $\mathcal{C}$.

Theorem 6.6 — Chart-Invisible Does Not Imply Primitive-Unavailable

If

$$\text{Path}_{\mathcal{C}}(x, y) = \emptyset,$$

it does not follow, in general, that

$$\text{Path}_R(x, y) = \emptyset.$$

Proof The inclusion

$$\text{Path}_{\mathcal{C}}(x, y) \subseteq \text{Path}_R(x, y)$$

is one-directional. The empty subset of a set does not imply that the containing set is empty. Additional path-completeness assumptions are required.

Corollary 6.7 — Path Absence Requires Completeness

Only under a declared path-completeness condition can chart-visible absence imply primitive absence for the relevant task.

6.8 Failure Mode Taxonomy

The following are distinct.

1. Primitive unavailability: no primitive path exists, if a primitive path structure is declared.
2. Chart invisibility: a path is not visible in the current chart.
3. Atlas absence: no chart in the atlas exposes the path.
4. Task inadmissibility: a visible branch fails the task-admission region.
5. Calibration unavailability: comparable branch data are not available.
6. Calibration instability: comparable branches are available but fail calibration tolerance.
7. Regressive learning: a branch is admitted but evaluated as worsening.
8. Boundary exit: a branch lands outside the proper organization layer.

These failure modes must not be conflated.

7. No-Go Results for Trace-Admission and Learning Direction

Theorem 7.1 — No Evaluator, No Learning Direction

Let p be a branch. If no evaluator mode or equivalent loss mode is declared for a task τ , then p cannot be classified as improving, neutral, worsening, or incomparable relative to τ .

Proof Each classification requires the outcome value $E_\tau(p)$. If no evaluator or equivalent loss mode is declared, this value does not exist. Hence none of the outcome predicates is defined.

Corollary 7.2 — Branch Is Not Learning Direction

Available branching alone is not learning direction. Learning direction requires task-relative branch evaluation.

Theorem 7.3 — No Evaluator, No Path Trace Admission

If no evaluator mode or equivalent loss/admission mode is declared for τ , then no branch is τ -trace-admissible.

Proof Trace-Admission requires

$$E_\tau(p) \in W_\tau^{\text{adm}}.$$

Without an evaluator or equivalent loss/admission mode, the left-hand side is undefined. Therefore the trace-admission condition cannot be satisfied.

Theorem 7.4 — No Dangling Path Trace Step

A dangling transition attempt

$$(\mathcal{C}, \psi, *)$$

cannot be a step in a learning path trace.

Proof A learning path trace is a finite landed branch. Every step in a branch has a source chart, an admissible transition, and a target chart. A dangling attempt has no target chart and is therefore not a landed step. Hence it cannot be a path trace step.

Theorem 7.5 — Non-Admitted Evaluation No-Go

Let $p \in X_\tau$ be an evaluated branch. If

$$E_\tau(p) \notin W_\tau^{\text{adm}},$$

then p is not a τ -learning path trace.

Proof A τ -learning path trace must have evaluation in the admissible region. Since $E_\tau(p) \notin W_\tau^{\text{adm}}$, the branch is not trace-admissible and hence not a learning path trace.

Theorem 7.6 — Worsening Can Be Learning

Let p be a τ -learning path trace. If

$$E_\tau(p) \in W_\tau^-,$$

then p is learning and is regressive learning.

Proof The branch is a learning path trace by hypothesis. Membership in W_τ^- makes it worsening by Definition 4.6, and regressive by Definition 4.8.

Theorem 7.7 — Constant Outcome No Directional Classification Beyond Neutrality

Suppose the evaluator outcome classifies every trace-admissible branch in a class $X \subseteq X_\tau$ as neutral:

$$E_\tau(x) \in W_\tau^0 \quad \forall x \in X.$$

Then no learning path trace in X is improving or worsening unless the outcome classes overlap.

Proof Improving and worsening require membership in W_τ^+ and W_τ^- , respectively. If every evaluated branch lies in W_τ^0 , and the outcome classes are disjoint, none lies in W_τ^+ or W_τ^- . If the classes overlap, the conclusion depends on the declared overlap convention.

8. Proper-Layer Conditions and Trace Threshold Policies

8.1 Trace Threshold Policy

Let

$$p : \mathcal{C}_0 \xrightarrow{\phi_0} \mathcal{C}_1 \xrightarrow{\phi_1} \dots \xrightarrow{\phi_{n-1}} \mathcal{C}_n$$

be a finite branch. A τ -threshold policy is a rule

$$\Theta_\tau : \text{Br}_{\mathfrak{A}}^{<\infty} \rightarrow \bigsqcup_{n \geq 0} \prod_{i=0}^n Z_{\mathcal{C}_i}^\circ$$

assigning to each branch p a threshold schedule

$$\Theta_\tau(p) = (\theta_0, \dots, \theta_n), \quad \theta_i \in Z_{\mathcal{C}_i}^\circ.$$

The schedule is part of the declared task policy. It is not chosen after a branch has already been judged proper.

A threshold policy may depend on declared branch features, including:

$$|p| = \text{branch length},$$

$$d_{\mathfrak{A}}(\text{src } p, \text{tgt } p) = \text{minimal atlas-distance between endpoints},$$

$$K_\tau(p) = \text{declared branch complexity or cost},$$

$$\text{Var}_\tau(\text{src } p, \text{tgt } p) = \text{number of task-distinct available branch variants between the endpoints}.$$

When the target zero-boundary orders have an agreed orientation in which larger thresholds are stricter, a threshold policy is called **progressive** along p when

$$\theta_i \preceq \theta_{i+1} \quad \text{after transporting thresholds along the declared transition maps, whenever such transport is defined.}$$

It is called **relaxing** along p when the transported inequalities are reversed. It is called **adaptive** when no monotonicity convention is imposed and the schedule is determined by a declared policy functional, for example

$$\theta_i = H_\tau(K_\tau(p_{\leq i}), \text{Var}_\tau(p_{\leq i})),$$

where $p_{\leq i}$ is the prefix ending at $\mathcal{C}_i$.

No universal monotonicity direction is assumed. In some tasks increasing complexity warrants stricter thresholds; in others maintaining viability across a long or unstable branch warrants relaxation. The monotonicity convention is task-declared.

8.2 Entropy-Like Readings of Complexity

If a branch-complexity functional K_τ is derived from a probability distribution, coding length, or multiplicity of admissible variants, then it may admit an entropy-like interpretation. This paper does not assume entropy, probability, thermodynamics, or information theory. Entropy-like quantities are optional realizations of K_τ , not part of the base theory.

8.3 Proper Learning Path Trace

A τ -learning path trace

$$p : \mathcal{C}_0 \rightarrow \mathcal{C}_1 \rightarrow \dots \rightarrow \mathcal{C}_n$$

is proper relative to a threshold policy Θ_τ when

$$\Theta_\tau(p) = (\theta_0, \dots, \theta_n)$$

and

$$\text{Org}_{\theta_i}^\circ(\mathcal{C}_i) \neq \emptyset \quad \text{for every } i.$$

Theorem 8.4 — Boundary-Collapsed Target No Proper Learning

If a branch lands in a chart $\mathcal{D}$ whose proper organization layer is empty at every proper threshold, then the branch is not a proper learning path trace.

Proof Proper learning requires each chart along the path trace, including the target chart, to have nonempty proper organization layer at the threshold assigned by the declared threshold policy. If no such threshold exists for $\mathcal{D}$, the condition fails.

Theorem 8.5 — Constant-Resolution Target No Proper Learning

If a branch lands in a constant-resolution chart $\mathcal{D}$, and constant-resolution charts have empty proper organization layer by the charted organization no-go, then the branch is not a proper learning path trace.

Proof This is Theorem 8.4 applied to a constant-resolution chart.

Theorem 8.6 — Proper Learning Is Stronger Than Trace-Admission

A branch may be trace-admissible without being proper under the declared threshold policy.

Proof Trace-admission requires evaluation admission. Properness additionally requires nonempty proper organization layers at the thresholds assigned by the declared threshold policy along the branch. These are distinct conditions. Therefore trace-admission alone does not imply properness.

9. Failure, Non-Resolution, and Refinement

9.1 Primordial Non-Resolution

A task τ has primordial non-resolution in an atlas $\mathfrak{A}_R$ when direct endpoint or boundary-collapsed charts do not resolve τ .

If $\mathbf{B}_R \subseteq \mathbf{C}_R$ is the class of boundary-collapsed and endpoint-only charts, then primordial non-resolution for τ means

$$\mathbf{B}_R \cap \text{Res}(\tau) = \emptyset.$$

9.2 Local Failure

A local failure of a branch or step may occur by non-admission, improperness, calibration instability, or regressive evaluation.

These cases should not be conflated.

9.3 Refinement Without Local Failure

A learning path trace may be improving or neutral even if the source chart has not locally failed. Such a transition is refinement, stabilization, or compression rather than repair.

Thus local failure is not required at every learning step.

Theorem 9.4 — Primordial Non-Resolution Is Required for Task-Forced Nontrivial Learning

If a task τ is resolved by an endpoint-only or boundary-collapsed chart in $\mathfrak{A}_R$, then τ does not force nontrivial charted learning.

Proof If τ is resolved by an endpoint-only or boundary-collapsed chart, then there exists a resolving chart for τ that does not require a nontrivial proper chart path. Therefore the task can be satisfied without forcing a nontrivial learning path trace. This does not imply that no learning path trace for τ can occur; it implies only that such a path trace is not forced by the task.

Corollary 9.5 — Local Failure Is Not Necessary at Every Step

A learning path trace may contain improving or neutral transitions after the original task gap is already partially resolved.

10. Delineation, Refinement, and Coarsening

10.1 Chart Delineation Relation

For a chart $\mathcal{C}$, define the chart-delineation relation

$$D_{\mathcal{C}} \subseteq R \times R$$

by

$$(r, s) \in D_{\mathcal{C}}$$

when r and s both lie in the chart carrier of $\mathcal{C}$ and are separated by at least one chart-induced face or threshold cut.

Equivalently, r and s are delineated by $\mathcal{C}$ when they are not identified by the relevant declared chart kernel or cut relation.

10.2 Refinement

A transition

$$\phi : \mathcal{C} \rightarrow \mathcal{D}$$

is delineation-refining when

$$D_{\mathcal{C}} \subsetneq D_{\mathcal{D}}.$$

10.3 Coarsening

A transition

$$\phi : \mathcal{C} \rightarrow \mathcal{D}$$

is delineation-coarsening when

$$D_{\mathcal{D}} \subsetneq D_{\mathcal{C}}.$$

10.4 Lateral Recharting

A transition is lateral when neither

$$D_{\mathcal{C}} \subseteq D_{\mathcal{D}}$$

nor

$$D_{\mathcal{D}} \subseteq D_{\mathcal{C}}$$

holds.

Theorem 10.5 — Refinement Does Not Imply Improvement

A delineation-refining transition need not be improving for a task τ .

Proof Refinement is defined by inclusion of chart-delineation relations. Improvement is defined by membership of the evaluated branch outcome in W_{τ}^{+} . The definition of refinement imposes no condition on E_{τ} . Therefore refinement alone does not imply improvement.

Theorem 10.6 — Coarsening Does Not Imply Worsening

A delineation-coarsening transition need not be worsening for a task τ .

Proof Coarsening is defined by reduction of the delineation relation. Worsening is defined by membership in W_τ^- . If the lost delineations are irrelevant to τ , the evaluator may classify the branch as neutral or improving. Hence coarsening alone does not imply worsening.

11. Compression of Learning Path Traces

11.1 Path Trace Compression

Let

$$L_\tau : \mathcal{C}_0 \rightarrow \mathcal{C}_1 \rightarrow \dots \rightarrow \mathcal{C}_n$$

be a τ -learning path trace.

A path trace compression is a chart $\mathcal{K}$ together with comparison data showing that $\mathcal{K}$ preserves declared task-visible distinctions or outcomes from L_τ .

11.2 Task-Lossless Compression

A path trace compression $\mathcal{K}$ is lossless for τ when

$$\text{Loss}_\tau(L_\tau, \mathcal{K}) = 0.$$

The loss function must be declared as part of the task's loss mode.

11.3 Lossy Compression

A path trace compression is lossy for τ when

$$\text{Loss}_\tau(L_\tau, \mathcal{K}) > 0$$

in the declared loss ordering.

11.4 Compression Is Task-Relative

A compression may be lossless for τ and lossy for another task σ .

Thus compression is not absolute unless the task family is universal over the relevant chart-visible distinctions.

Theorem 11.5 — Task-Losslessness Does Not Imply Universal Losslessness

Let $\mathcal{K}$ be a path trace compression such that

$$\text{Loss}_\tau(L_\tau, \mathcal{K}) = 0.$$

If there exists a task σ and a distinction in D_σ not preserved by $\mathcal{K}$, then $\mathcal{K}$ is not universally lossless.

Proof Universal losslessness requires preservation of all declared task-visible distinctions. The hypothesis states that $\mathcal{K}$ fails to preserve a σ -visible distinction. Therefore universal losslessness fails even though τ -losslessness holds.

12. Absolute Losslessness Boundary

12.1 Complete Task Family

A task family $\mathcal{T}$ is complete for a chart $\mathcal{C}$ when every chart-visible delineation in $D_{\mathcal{C}}$ appears in D_{τ} for some $\tau \in \mathcal{T}$.

12.2 Absolutely Lossless Compression

A compression $\mathcal{K}$ of $\mathcal{C}$ is absolutely lossless relative to $\mathcal{T}$ when it is lossless for every $\tau \in \mathcal{T}$.

12.3 Faithful Complete Task Family

A complete task family $\mathcal{T}$ is faithful for $\mathcal{C}$ when each task $\tau \in \mathcal{T}$ has a faithful loss mode for the delineations D_{τ} it demands.

Theorem 12.4 — Proper Sub-Chart Absolute Losslessness Boundary

Let $\mathcal{K} \subsetneq \mathcal{C}$ be a proper sub-chart. If $\mathcal{T}$ is faithful and complete for $\mathcal{C}$, and $\mathcal{K}$ fails to preserve some chart-visible delineation in $D_{\mathcal{C}}$, then no compression into $\mathcal{K}$ is absolutely lossless relative to $\mathcal{T}$.

Proof Completeness of $\mathcal{T}$ means the failed delineation is demanded by some task $\tau \in \mathcal{T}$. Faithfulness of the loss mode for τ means zero loss for τ implies preservation of every delineation demanded by τ . Since $\mathcal{K}$ fails one such delineation, the corresponding τ -loss cannot be zero. Absolute losslessness requires zero loss for every task in $\mathcal{T}$. Therefore compression into $\mathcal{K}$ cannot be absolutely lossless.

Corollary 12.5 — Task-Relative Losslessness Is the Proper Form

A proper sub-chart may be lossless for one task but not for a faithful complete task family.

Theorem 12.6 — Losslessness Is Monotone in the Task Family

Let $\mathcal{T}_1 \subseteq \mathcal{T}_2$. If a compression is lossless for every task in $\mathcal{T}_2$, then it is lossless for every task in $\mathcal{T}_1$. The converse need not hold.

Proof The forward implication follows because every task in $\mathcal{T}_1$ is also in $\mathcal{T}_2$. For the converse, take any compression that preserves the delineations demanded by $\mathcal{T}_1$ but fails a delineation demanded by some task in $\mathcal{T}_2 \setminus \mathcal{T}_1$. Such a compression is lossless for $\mathcal{T}_1$ but not for $\mathcal{T}_2$.

12.7 Subsumption Boundary

A proper chart can approach losslessness for broader and broader task families only by preserving more of the parent chart's delineations. If it preserves all parent-chart delineations for a faithful complete task family, it is equivalent to the parent presentation for those tasks.

13. Path Trace Identity and Path Dependence

Theorem 13.1 — Endpoint Equality Does Not Imply Path Trace Equality

There may exist two path traces

$$L : \mathcal{C}_0 \rightarrow \cdots \rightarrow \mathcal{C}_n$$

and

$$L' : \mathcal{C}_0 \rightarrow \cdots \rightarrow \mathcal{C}_n$$

with the same source and target but different intermediate charts, transition sequences, or evaluation records. Such path traces are endpoint-equal but not path trace-equivalent.

Proof Path Trace equivalence requires declared identity of chart sequence, transition sequence, and task-evaluation records. Endpoint equality identifies only the first and last charts. Therefore endpoint equality does not entail path trace equivalence.

Corollary 13.2 — Same Endpoint, Different Learning

Two learning processes may reach the same endpoint chart while representing different learning histories.

Corollary 13.3 — Endpoint Compression Loses Learning History

A representation that records only the source and target charts of a path trace cannot recover the path trace unless the intermediate transitions and evaluations are reconstructible from the endpoints.

14. Learning Processes

14.1 Learning Process

A learning process is a family of τ -learning path traces indexed by tasks, stages, or declared contexts.

Formally, a learning process may be written as

$$\mathcal{L} = \{L_{\tau_i}^{(j)} : i \in I, j \in J_i\}.$$

14.2 Single-Task Process

A single-task learning process is a sequence of path traces all evaluated under the same task τ .

14.3 Multi-Task Process

A multi-task learning process is a sequence or family of path traces evaluated under more than one task.

14.4 Transfer

A branch or path trace transfers from task τ to task σ only when the branch lies in the shared evaluation domain

$$p \in X_\tau \cap X_\sigma.$$

It transfers from τ to σ when chart changes admitted under τ also preserve or improve the evaluator outcome under σ .

Transfer requires both task modes to be declared. It is not automatic.

Theorem 14.5 — Task-Local Improvement Does Not Imply Transfer

If a path trace is improving for τ , it need not be improving for σ .

Proof Improvement for τ is defined by membership of the path trace outcome in W_τ^+ . Improvement for σ is defined using E_σ and W_σ^+ . No definition forces these evaluators or outcome classes to agree.

15. Update and Recharting

15.1 Chart Update

A chart update is a transition attempt intended to replace or modify a chart.

It becomes learning only when it appears as an admitted branch in a path trace.

15.2 Recharting

Recharting is a chart update in which the target chart has different face-readout maps, different induced delineations, or different zero-boundary resolution structure.

15.3 Refinement Update

A refinement update is a recharting transition that is delineation-refining.

15.4 Coarsening Update

A coarsening update is a recharting transition that is delineation-coarsening.

15.5 Recomputation Versus Learning

A chart update that recomputes a target chart without a task evaluator is not learning in the present framework.

Theorem 15.6 — Update Is Not Learning Without Admission

A chart update that is not trace-admissible for any declared task is not a learning path trace.

Proof A learning path trace is an admitted branch recorded as a path trace. If no task makes the update or resulting branch trace-admissible, then the update cannot be a learning path trace.

16. Recursive Path Trace Update

16.1 Path-Trace-Induced Update

A path-trace-induced update is a rule

$$U_\tau : \text{Tr}_\tau^{\text{adm}} \rightarrow \text{Upd}_\tau$$

assigning update data to admitted path traces. The update data may specify a new chart, a modified atlas, or a modified available step field.

16.2 Step-Field Update

A step-field update has the form

$$U_\tau(L) : \text{Step}_{\mathfrak{A}}^{(n)}(\mathcal{C}_n) \rightarrow \text{Step}_{\mathfrak{A}}^{(n+1)}(\mathcal{C}_n),$$

where L is a path trace ending at $\mathcal{C}_n$.

16.3 Recursive Learning

A learning process is recursive when at least one admitted path trace changes the later chart, atlas, or available step field from which subsequent branches are drawn.

Thus recursion is not merely the existence of a finite path. It is the dependence of later branch availability on earlier trace-admitted branches.

Theorem 16.4 — Finite Sequence Alone Does Not Imply Recursion

A finite path trace sequence with no path-trace-induced update to charts, atlas, or available step fields is sequential but not recursive in the sense of Definition 16.3.

Proof Definition 16.3 requires a dependence of later branch availability on an earlier admitted path trace. A finite sequence without such an update lacks that dependence.

16.5 Recursive Compression

A compression is recursive when the compressed path trace becomes part of the chart data or update data used to evaluate later branch fields. In this case the result of prior learning changes the presentation in which later learning occurs.

17. Worked Example: Quantitative Branching, Calibration, Compression, and Recursion

17.1 Chart Field

Let $\mathfrak{A}_R$ contain charts $\mathcal{C}_0, \mathcal{C}_1, \mathcal{C}_2, \mathcal{C}_3$. From $\mathcal{C}_0$, suppose the task-visible one-step branches are

$$p_1 : \mathcal{C}_0 \rightarrow \mathcal{C}_1, \quad p_2 : \mathcal{C}_0 \rightarrow \mathcal{C}_2.$$

Suppose also that both $\mathcal{C}_1$ and $\mathcal{C}_2$ admit a later transition to $\mathcal{C}_3$:

$$q_1 : \mathcal{C}_1 \rightarrow \mathcal{C}_3, \quad q_2 : \mathcal{C}_2 \rightarrow \mathcal{C}_3.$$

Thus the two length-two branches

$$P_1 = p_1 q_1, \quad P_2 = p_2 q_2$$

share source and endpoint but have different path traces.

17.2 Evaluator and Admission

Let the task evaluator have outcome space $W_\tau = [-1, 1]$, outcome classes

$$W_\tau^+ = (0, 1], \quad W_\tau^0 = \{0\}, \quad W_\tau^- = [-0.75, 0),$$

and admissible region

$$W_\tau^{\text{adm}} = [-0.75, 1].$$

Declare

$$E_\tau(p_1) = 0.8, \quad E_\tau(p_2) = -0.4,$$

so both one-step branches are admitted, but p_1 is improving and p_2 is regressive. Now declare

$$E_\tau(P_1) = 0.6, \quad E_\tau(P_2) = 0.2.$$

Both full branches are admitted and both land at $\mathcal{C}_3$, but their path trace outcomes differ.

17.3 Calibration

Let $P_1 \sim_\tau P_2$ be declared task-comparable. Let

$$\text{CalLoss}_\tau(P_1, P_2) = |E_\tau(P_1) - E_\tau(P_2)| = 0.4.$$

If the tolerance is 0.5, the pair is calibration-stable. If the tolerance is 0.25, the same pair is calibration-failed. This shows that calibration is relative to a declared comparison tolerance, not a separate path type.

17.4 Compression

Let a compression K of the path trace P_1 preserve the task-visible delineations for τ with

$$\text{Loss}_\tau(K) = 0.$$

but fail a second task σ with

$$\text{Loss}_\sigma(K) = 0.3.$$

Then K is τ -lossless but not lossless for $\{\tau, \sigma\}$. This illustrates Theorem 12.6.

17.5 Recursive Update

Suppose the admitted path trace P_1 updates the later step field at $\mathcal{C}_3$ by adding a new available branch $r : \mathcal{C}_3 \rightarrow \mathcal{C}_4$. Then

$$U_\tau(P_1) : \text{Step}^{(0)}(\mathcal{C}_3) \rightarrow \text{Step}^{(1)}(\mathcal{C}_3)$$

with

$$r \in \text{Step}^{(1)}(\mathcal{C}_3), \quad r \notin \text{Step}^{(0)}(\mathcal{C}_3).$$

This is recursion in the formal sense of the document: an admitted path trace changes the available branch field for later learning.

17.6 Trace Threshold Policy in the Example

Let branch length, branch complexity, and endpoint-variant count be declared as

$$|P_1| = |P_2| = 2,$$

$$K_\tau(P_1) = 1.2, \quad K_\tau(P_2) = 1.8,$$

and

$$\text{Var}_\tau(\mathcal{C}_0, \mathcal{C}_3) = 2.$$

Assume every chart in the example has zero-boundary interval $[0, 1]$, and assume that larger thresholds are stricter. A progressive threshold policy may be declared by

$$\theta_i(P) = 0.35 + 0.05i + 0.05K_\tau(P).$$

Then

$$\Theta_\tau(P_1) = (0.41, 0.46, 0.51),$$

while

$$\Theta_\tau(P_2) = (0.44, 0.49, 0.54).$$

The higher-complexity branch receives a stricter schedule. If a viability-preserving task instead declares a relaxing policy, one may set

$$\theta_i(P) = 0.60 - 0.05i,$$

so later stages of the same path trace are evaluated against broader thresholds. A relaxing schedule does not mean weaker theory discipline; it means the task has declared progressive viability rather than progressive strictness as the governing convention.

Both policies are admissible only because their monotonicity convention and dependence on branch features are declared. Neither is forced by the base theory.

18. Worked Example: Branching, Calibration, and Path Tracing

18.1 Charts and Available Steps

Let an atlas contain four charts

$$\mathcal{C}_0, \mathcal{C}_1, \mathcal{C}_2, \mathcal{C}_3.$$

Assume the following transitions are in $\mathbf{T}_R$:

$$\phi_a : \mathcal{C}_0 \rightarrow \mathcal{C}_1, \quad \phi_b : \mathcal{C}_0 \rightarrow \mathcal{C}_2,$$

and

$$\psi_a : \mathcal{C}_1 \rightarrow \mathcal{C}_3, \quad \psi_b : \mathcal{C}_2 \rightarrow \mathcal{C}_3.$$

Thus two branches from $\mathcal{C}_0$ re-merge at $\mathcal{C}_3$:

$$p_a : \mathcal{C}_0 \xrightarrow{\phi_a} \mathcal{C}_1 \xrightarrow{\psi_a} \mathcal{C}_3,$$

and

$$p_b : \mathcal{C}_0 \xrightarrow{\phi_b} \mathcal{C}_2 \xrightarrow{\psi_b} \mathcal{C}_3.$$

18.2 Task Evaluation

Let

$$W_\tau = \{-1, 0, +1\}$$

with

$$W_\tau^{\text{adm}} = W_\tau, \quad W_\tau^+ = \{+1\}, \quad W_\tau^0 = \{0\}, \quad W_\tau^- = \{-1\}.$$

Declare

$$E_\tau(p_a) = +1, \quad E_\tau(p_b) = -1.$$

Both branches are admitted learning path traces, since both evaluations lie in W_τ^{adm} . The branch p_a is improving. The branch p_b is worsening.

Thus two branches can share source and endpoint while carrying different learning outcomes.

18.3 Calibration

Declare $(p_a, p_b) \in \text{CalDom}_\tau$, and let

$$\text{CalLoss}_\tau(p_a, p_b) = 2.$$

If the calibration-admissible region is

$$L_\tau^{\text{cal}} = \{0, 1\},$$

then this branch pair fails calibration.

This does not mean no branch exists. It means the task-comparable available branches are unstable under the declared calibration tolerance.

18.4 Dangling Attempt

Suppose there is also an attempted transition

$$(\mathcal{C}_3, \omega, *).$$

Since this attempt has no target chart, it is not a landed step and cannot enter a path trace.

19. Worked Example: Same Endpoint, Different Path Trace

19.1 Two Paths

Let

$$\phi : \mathcal{C}_0 \rightarrow \mathcal{C}_2$$

and

$$\psi_0 : \mathcal{C}_0 \rightarrow \mathcal{C}_1, \quad \psi_1 : \mathcal{C}_1 \rightarrow \mathcal{C}_2$$

be admitted transitions.

Then there are two endpoint-equal branches:

$$L : \mathcal{C}_0 \xrightarrow{\phi} \mathcal{C}_2,$$

and

$$L' : \mathcal{C}_0 \xrightarrow{\psi_0} \mathcal{C}_1 \xrightarrow{\psi_1} \mathcal{C}_2.$$

19.2 Evaluations

Let

$$E_\tau(L) = 0,$$

while

$$E_\tau(L') = +1.$$

Both path traces have the same endpoints, but their evaluation records differ.

Hence endpoint equality does not imply learning equality.

20. Worked Example: Compression and Losslessness

Let

$$L_\tau$$

be a learning path trace, and let $\mathcal{K}$ be a compressed chart.

Suppose $\mathcal{K}$ preserves all distinctions in D_τ , so

$$\text{Loss}_\tau(L_\tau, \mathcal{K}) = 0.$$

Let σ be another task with distinction demand

$$D_\sigma \not\subseteq D_\tau.$$

If $\mathcal{K}$ fails to preserve a distinction in

$$D_\sigma \setminus D_\tau,$$

then

$$\text{Loss}_\sigma(L_\tau, \mathcal{K}) > 0.$$

Thus $\mathcal{K}$ is lossless for τ but lossy for σ .

21. Motivating Realizations

21.1 Status

The following are motivating interpretations only. They are not part of the formal theory.

21.2 Brain

A biological brain may be studied as a physical realization of recursive chart learning if neural-state transitions are represented as admissible chart transitions and if tasks/evaluators/calibration modes are physically or behaviorally declared.

This paper does not claim that it has modeled brains.

21.3 Language

Language may be studied as a social-symbolic realization of recursive chart learning if utterances, meanings, and communicative repair are represented as charted transitions preserving task-visible delineations.

This paper does not claim that it has modeled language.

21.4 Large Language Models

An LLM may be studied as a computational realization of recursive chart learning if prompt-conditioned generation is represented as chart transition under declared evaluator modes.

This paper does not claim that it has modeled LLMs.

21.5 Shared Abstract Loop

Brains, language, and LLMs may instantiate a common abstract loop:

input $\rightarrow$ chart $\rightarrow$ branch field $\rightarrow$ evaluation $\rightarrow$ admitted branch $\rightarrow$ path trace $\rightarrow$ compression or recharting.

The present paper supplies the formal vocabulary for that comparison, not a proof of common realization.

22. Self-Limitation

22.1 No Unobserved Learning Claim

This framework does not claim that every physical, biological, linguistic, or computational change is learning. Only admitted, trace-admitted, task-evaluated chart-visible branching is learning in this theory.

22.2 No Improvement Guarantee

The framework does not guarantee that learning improves task resolution.

Improvement, neutrality, worsening, and incomparability are all possible outcomes.

22.3 No Global Path Absence From Local Invisibility

The framework does not infer primitive path absence from chart-visible path absence unless a path-completeness condition is declared.

22.4 No Global Losslessness

The framework does not allow a proper sub-chart compression to be treated as globally lossless unless the complete task family is preserved.

22.5 No Endpoint-Only Identity

The framework does not identify learning processes by endpoints alone.

22.6 No Psychology Claim

The framework does not define cognitive learning. It defines an abstract organization-theoretic structure that later cognitive theories may or may not realize.

23. Claims and Non-Claims

23.1 Claims

This paper claims:

1. learning can be formalized as observer-relative path tracing of admitted branches through a chart-visible path field;
2. evaluation is not oracular: it applies to chart-visible available branches or declared task data;
3. calibration is branch-stability under task evaluation, not a separate path type;
4. path traces record only landed and admitted branches;
5. learning does not imply improvement;
6. worsening is an explicit possible outcome of learning;

7. dangling attempts cannot be path trace steps;
8. local path invisibility does not imply primitive path absence;
9. endpoint equality does not imply path trace equality;
10. compression is task-relative;
11. absolute losslessness by a proper sub-chart is blocked under complete task separation;
12. brain, language, and LLM examples are possible motivations, not formal realizations.

23.2 Non-Claims

This paper does not claim:

1. that all change is learning;
 2. that all learning improves;
 3. that every transition attempt becomes a path step;
 4. that failed attempts are impossible;
 5. that failed attempts are trace-admitted learning steps;
 6. that evaluators are new primitives;
 7. that calibration is an external validation mechanism;
 8. that chart invisibility means primitive nonexistence;
 9. that cognitive learning has been modeled;
 10. that physical dynamics has been modeled;
 11. that language or LLMs have been formally derived;
 12. that reality is learning.
-

24. Conclusion

Recursive Chart Learning Theory defines learning as observer-relative path tracing of admitted branches through a chart-visible path field. It separates transition attempts from landed steps, landed steps from admitted branches, and admitted branches from improving outcomes.

A learning path trace cannot contain a dangling arrow. Every trace-admitted branch has admissible endpoints and satisfies the evaluator’s admission condition. Yet that same branch may still be evaluated as worsening. This distinction separates success as path-completion from success as task-improvement.

The theory is path-sensitive. Branches may split and re-merge. The same endpoint chart may be reached by different path traces, and those path traces may carry different evaluation histories. Learning is therefore not reducible to endpoint state.

Calibration is not a separate path type. It is the stability of task-evaluation over task-comparable branches in the available branch field. Calibration unavailability, calibration instability, task inadmissibility, path invisibility, and primitive path absence are different conditions.

Compression is task-relative. A chart may compress a learning path trace without loss for one task while losing distinctions relevant to another. Absolute losslessness is not available to a proper sub-chart unless the complete task-visible structure of the parent chart is preserved.

The central lesson is:

Learning is not successful update. Learning is admitted, evaluated, trace-admitted branching.

AI-assistance disclosure

This manuscript was developed with AI-assisted drafting and review support. The author directed the research programme, selected and revised the mathematical and methodological content, and is responsible

for all claims and errors.